TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN GAME CARO

GIÁO VIÊN HƯỚNG DẪN:

TS. Trương Toàn Thịnh

NHÓM SINH VIÊN THỰC HIỆN

Vũ Đình Anh Kiệt (MSSV: 25127400)

Huỳnh Phúc Nguyên (MSSV: 25127106)

Nguyễn Kim Phúc (MSSV: 25127464)

Nguyễn Trần Hoàng Quân (MSSV: 25127231)

Huỳnh Đình Khương (MSSV: 25127396)

Tháng 12/2025

fit@hcmus

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN CUỐI KÌ

CỜ CARO

THÔNG TIN CHUNG

Người hướng dẫn:

– TS. Trương Toàn Thịnh (Khoa Công nghệ Thông tin)

Nhóm sinh viên thực hiện:

1. Vũ Đình Anh Kiệt (MSSV: 25127400)
2. Huỳnh Phúc Nguyên (MSSV: 25127106)
3. Nguyễn Kim Phúc (MSSV: 25127464)
4. Nguyễn Trần Hoàng Quân (MSSV: 25127231)
5. Huỳnh Đình Khương (MSSV: 25127396)

Loại đề tài: Đồ án

Thời gian thực hiện: Từ *6/10/2025* đến *22/12/2025*

MỤC LỤC

MỤC LỤC **3**

A. THÔNG TIN CHUNG **7**

1. GIỚI THIỆU **7**

1.1 TỔNG QUAN ĐỀ TÀI **7**

1.2 MỤC TIÊU **7**

2. CẤU TRÚC VÀ TỔ CHỨC CHƯƠNG TRÌNH **7**

2.1 TỔ CHỨC KIẾN TRÚC TRÒ CHƠI **8**

2.1.1 Tầng quản lý tài nguyên và toàn cục 8

2.1.2 Tầng giao diện và điều khiển 8

2.1.3 Tầng logic trò chơi 9

2.2 LUỒNG XỬ LÝ DỮ LIỆU **9**

B. NHÓM CÂU LỆNH HIỂN THỊ **11**

1. NHÓM BIẾN, ĐỐI TƯỢNG VÀ THÔNG SỐ **11**

2. NHÓM HÀM **12**

2.1 CHI TIẾT CÁCH THỨC HOẠT ĐỘNG **13**

2.1.1 void Setting::draw() 13

2.1.2 void Menu::drawMenu() 14

2.1.3 void Menu::drawPopUpMenu() 14

2.1.4 `void Cell::draw()` 15

2.1.5 `void Board::setHighlight()` 15

2.1.6 `void Board::draw()` 15

2.1.7 `void Game::drawResult()` 16

2.1.8 Hiệu ứng vẽ quân cờ 16

2.1.9 Hiệu ứng chuyển cảnh xé giấy 17
2.1.10 Hiệu ứng chiến thắng và hòa 17
C. NHÓM CÂU LỆNH LOGIC VÀ XỬ LÝ 17
1. NHÓM BIẾN, ĐỐI TƯỢNG VÀ THÔNG SỐ 17
2. NHÓM HÀM LOGIC VÀ XỬ LÝ 20
2.1 KHỞI TẠO VÀ CÀI ĐẶT 20
2.2 XỬ LÝ INPUT VÀ ĐIỀU KHIỂN 21
2.3 XỬ LÝ LOGIC VÀ TRẠNG THÁI 22
2.4 TRẠNG THÁI GIAO DIỆN 24
2.5 CHI TIẾT CÁCH THỨC HOẠT ĐỘNG NHỮNG HÀM PHỨC TẠP 25
2.5.1 Khởi tạo và cài đặt 25
2.5.1.1 `SoundManager::SoundManager()` 25
2.5.1.2 `TextureManager::TextureManager()` 25
2.5.1.3 `Menu::createBox(...)` 25
2.5.1.4 `Menu::createMenu(...)` 26
2.5.1.5 `Board::createBoard()` 26
2.5.2 Xử lý input và điều khiển 27
2.5.2.1 `void Menu::EventController()` 27
2.5.2.2 `void Game::EventController()` 27
2.5.3 Xử lý logic và trạng thái 28
2.5.3.1 `void Board::moveCursor(const int& dx, const int& dy)` 28
2.5.3.2 `bool Board::placeMark(const char& mark)` 28
2.5.3.3 `bool Board::checkWin(const char& mark)` 28
2.5.3.4 `bool Board::checkDraw()` 29
2.5.3.5 `void Board::resetData()` 29
2.5.3.6 `void Game::switchPlayer()` 29

2.5.3.7 `void Game::resetData()` 30
2.5.3.8 `void Game::runGame()` 30
2.5.3.9 `void Bot::makeMove() & int Bot::minimax(...)` 30
2.5.3.10 void GameController::saveGame(bool) & loadGame(string) 31
2.5.3.11 Logic Extreme Game (Chế độ mở rộng) 31
2.5.4 Trạng thái giao diện 31
2.5.4.1 `void Button::setNormalState()` 32
2.5.4.2 `void Button::setHoveredState()` 32
2.5.4.3 `void Label::setNormalState()` 32
2.5.4.4 `void Label::setHoveredState()` 32
2.6 Thuật toán BOT đánh cờ Caro 33
2.6.1 Cấu trúc dữ liệu 33
2.6.2 Quy trình xử lý 33
2.6.2.1 Sàng lọc nước đi 33
2.6.2.2 Hàm đánh giá cục bộ 33
2.6.2.3 Thuật toán Minimax 33
2.7 Chế độ Extreme 34
2.7.1 Kiến trúc bàn cờ 34
2.7.2 Logic xử lý và hiển thị 34
2.8 Hệ thống save/load 34
2.8.1 Kỹ thuật quản lý file 34
2.8.2 Quy trình tuần tự hóa dữ liệu 35
2.8.3 Quy trình giải tuần tự hóa 35
D. MÔ PHỎNG CÁCH TRÒ CHƠI HOẠT ĐỘNG 35
1. LƯU ĐỒ TOÀN BỘ CHƯƠNG TRÌNH 35
2. MÀN HÌNH CHÍNH 36
3. MÀN HÌNH TRÒ CHƠI 38

4. MÀN HÌNH OPTIONS (TRONG MÀN HÌNH TRÒ CHƠI) 42

5. CHẾ ĐỘ EXTREME 42

6. Màn hình HELP và ABOUT 44

A. THÔNG TIN CHUNG

1. GIỚI THIỆU

1.1 TỔNG QUAN ĐỀ TÀI

Đồ án này được thực hiện nhằm mục đích hiện thực hóa trò chơi cờ Caro hợp các kỹ thuật lập trình C++ và cấu trúc dữ liệu cơ bản như mảng hai chiều.

Mặc dù yêu cầu cơ bản của đồ án là xây dựng trên môi trường console, nhóm chúng em đã quyết định vận dụng khả năng tự nghiên cứu, học hỏi để nâng cấp trò chơi lên giao diện đồ họa, sử dụng thư viện **SFML** (phiên bản 2.6.2). Nhằm mục đích:

- Tạo ra một giao diện người dùng trực quan, sinh động và gần gũi với thực tế hơn so với màn hình dòng lệnh.
- Cải thiện trải nghiệm người dùng thông qua việc điều khiển bằng bàn phím kết hợp phản hồi bằng hình ảnh và âm thanh.

1.2 MỤC TIÊU

Chương trình hướng tới việc xây dựng một hệ thống game hoàn chỉnh với các tính năng:

- **Gameplay:** Tuân thủ luật chơi Caro tiêu chuẩn: bàn cờ kích thước 15x15, thắng khi có 5 quân liên tiếp thỏa mãn theo đường thẳng/ngang/chéo.
- **Chế độ chơi:** Chế độ Classic bao gồm: chế độ người đấu với người (PvP) chế độ cho người đấu với máy (PvE) và chế độ Extreme.
- **Hệ thống Menu:** Tổ chức màn hình chính, màn hình Options và màn hình Settings, màn hình Load, Help, About.
- **Kỹ thuật lập trình:** Áp dụng triệt để lập trình hướng đối tượng để quản lý mã nguồn, tách biệt rõ ràng giữa xử lý logic, hiển thị và điều khiển. Bên cạnh đó các kĩ năng thu nhập đầu vào cũng quan trọng không kém.

2. CẤU TRÚC VÀ TỔ CHỨC CHƯƠNG TRÌNH

Thay vì sử dụng các hàm rời rạc và biến toàn cục đơn giản như hướng dẫn cơ bản, chương

trình của nhóm chúng em được tái cấu trúc theo mô hình hướng đối tượng kết hợp việc tự thiết kế các tài nguyên đồ họa, phục vụ cho việc hiển thị chương trình.

2.1 TỔ CHỨC KIẾN TRÚC TRÒ CHƠI

Hệ thống được chia thành 3 tầng xử lý chính:

2.1.1 Tầng quản lý tài nguyên và toàn cục

Đây là xương sống của toàn bộ chương trình, nơi khởi tạo và lưu trữ các tài nguyên dùng chung để tối ưu hóa bộ nhớ và thuận tiện trong việc sử dụng, sửa lỗi và phát triển chương trình sau này:

- **Global.cpp, Global.h:** Nơi khai báo các đối tượng duy nhất như `WINDOW`, `CARO`, và `GAME`.
- **Cấu trúc SoundManager:** Quản lý việc tải và phát nhạc nền cùng các hiệu ứng âm thanh.
- **Cấu trúc TextureManager:** Chịu trách nhiệm tải toàn bộ hình ảnh của bàn cờ, quân cờ và giao diện một lần duy nhất khi khởi chạy.

2.1.2 Tầng giao diện và điều khiển

Nhóm sử dụng tính chất kế thừa để quản lý hệ thống menu phức tạp:

- **Struct Menu:** Định nghĩa các thuộc tính cơ bản như danh sách nút, nhãn, và cơ chế điều hướng con trỏ.
- **Các lớp dẫn xuất:**
    + **MainScreen:** Quản lý menu chính với các tùy chọn NEW GAME, LOAD, SETTINGS, HELP, ABOUT, EXIT.
    + **GameOption:** Quản lý menu tạm dừng khi người chơi nhấn **ESC** trong màn hình trò chơi.
    + **GameOver:** Quản lý màn hình kết thúc ván chơi và điều hướng người chơi lựa chọn các lựa chọn sau khi ván chơi kết thúc.

+ **GameMode:** Cho phép lựa chọn chế độ chơi PvP hoặc PvE.
+ **GameSaver:** Giao diện lưu file.
+ **GameManager:** Giao diện quản lý/tải file save.
+ **PlayerNameInput:** Giao diện nhập tên người chơi.
+ **GameType:** Giao diện chọn loại game (Classic/Extreme).

2.1.3 Tầng logic trò chơi

Nơi xử lý các thuật toán cốt lõi của trò chơi Caro:

- **struct Game:** Là cấu trúc điều phối vòng lặp game, chuyển đổi lượt chơi và xử lý các sự kiện đầu vào của bàn phím trong ván đấu.
- **struct Board:**

  Quản lý dữ liệu mảng 2 chiều `cells[][]` đại diện cho bàn cờ 15x15.

  + Chứa thuật toán checkWin quan trọng: Kiểm tra chiến thắng theo các hướng sau mỗi nước đi.
  + Quản lý việc vẽ bàn cờ và con trỏ điều khiển.
- **struct Bot:** Chịu trách nhiệm tính toán nước đi thông minh cho máy.
- **struct ExtremeGame:** Quản lý logic cho chế độ chơi mở rộng (3x3 bàn cờ).
- **struct GameSaver:** Xử lý logic đọc/ghi file fstream.

2.2 LUỒNG XỬ LÝ DỮ LIỆU

- **Khởi động:** `main()` gọi `CARO.start()` → nạp tài nguyên → hiển thị menu chính.
- **Vào Game:** người dùng chọn chế độ → `GAME.start()` sẽ khởi tạo dữ liệu bàn cờ rỗng.
- **Vòng lặp Game:**
  + **Input:** Nhận tín hiệu phím **W/A/S/D/Enter** từ người chơi.

    + **Update:** `Board` cập nhật vị trí quân cờ, `checkWin()` kiểm tra trạng thái thắng thua.
    + **Render:** Xóa màn hình và vẽ lại bàn cờ cùng các hiệu ứng đồ họa.

– **Kết thúc:** Khi có kết quả, kích hoạt menu sau ván chơi để hỏi người dùng chơi tiếp, lưu ván chơi hay thoát.

B. NHÓM CÂU LỆNH HIỂN THỊ

1. NHÓM BIẾN, ĐỐI TƯỢNG VÀ THÔNG SỐ

Tên biến	Vị trí	Chức năng
`Window WINDOW`	Global.h, UI.h	Đối tượng cửa sổ hiển thị chính của chương trình.
`TextureManager TEXTURE`	Global.h, UI.h	Chứa tất cả các hình ảnh như Button, Board, XMark, OMark, XWinsBackground…
`FontManager FONT`	Global.h, UI.h	Chứa font chữ chính toàn bộ chương trình được tải từ file .ttf.
`Color COLOR`	Global.h, UI.h	Chứa các mã màu định nghĩa sẵn: PRIMARY, BACKGROUND, WHITE.
`sf::Sprite spriteMark`	Board.h	Hình ảnh hiển thị của quân X hoặc O tại một ô cụ thể.
`sf::RectangleShape board`	Board.h	Hình dạng và texture của bàn cờ.
`sf::RectangleShape highlight`	Board.h	Ô vuông viền màu dùng để đánh dấu vị trí con trỏ hiện tại trong bàn cờ trò chơi.
`sf::RectangleShape panel`	Menu.h, Setting.h	Khung nền của các menu hoặc bảng cài đặt.
`vector<Button> buttons`	Menu.h, Setting.h	Mảng chứa các nút bấm trên menu.

`vector<Label> labels`	Menu.h, Setting.h	Mảng chứa các dòng chữ của từng nút bấm trên menu.
`vector<sf::Sprite> icons`	Setting.h	Các icon biểu tượng trong phần Settings.
Các hằng số kích thước		
`WIDTH, HEIGHT`	UI.h	Kích thước cửa sổ (1920*1080).
`CELL_SIZE`	Board.h	Kích thước một ô cờ (50px).
`BOARD_PIXEL_SIZE`	Board.h	Kích thước hiển thị của cả bàn cờ (800px).
`START_X, START_Y`	Board.h	Tọa độ bắt đầu vẽ bàn cờ.
`buttonWidth, buttonHeight`	Menu.h	Kích thước mặc định của nút bấm menu (380*95).

2. NHÓM HÀM

Nhóm này chịu trách nhiệm vẽ, hiển thị hình ảnh, văn bản, nút bấm lên cửa sổ WINDOW.

Tên hàm	**Vị trí**	**Chức năng**
`void Setting::draw()`	Setting.cpp	Vẽ toàn bộ giao diện menu cài đặt gồm: nền, nút bấm, chữ.
`void Menu::drawMenu()`	Menu.cpp	Vẽ các nút và nhãn của một menu cơ bản.
`void Menu::drawPopUpMenu()`	Menu.cpp	Trong màn hình bàn cờ trò chơi, vẽ một lớp phủ mờ màu đen trước, sau đó vẽ menu

		Options đè lên.
`void Cell::draw()`	Board.cpp	Vẽ một ô cờ cụ thể.
`void Board::setHighlight()`	Board.cpp	Cập nhật vị trí và vẽ ô vuông highlight bao quanh con trỏ hiện tại trong bàn cờ trò chơi.
`void Board::draw()`	Board.cpp	Hàm vẽ tổng hợp của bàn cờ: vẽ khung bàn cờ, gọi lệnh vẽ cho từng ô cờ và vẽ highlight ô cờ.
`void Game::drawResult()`	Game.cpp	Vẽ bảng thông báo kết quả (X Wins, O Wins, hoặc Draw) khi game kết thúc.

2.1 CHI TIẾT CÁCH THỨC HOẠT ĐỘNG

2.1.1 `void Setting::draw()`

- **Vị trí:** File Setting.cpp.
- **Chức năng:** Vẽ toàn bộ giao diện của màn hình cài đặt, bao gồm nền bảng, các nút bấm và nhãn chữ.
- **Cách hoạt động:**
    + **Cập nhật trạng thái:** Hàm duyệt vòng lặp qua tất cả các tùy chọn (`numOptions`). Nó kiểm tra xem tùy chọn `i` có trùng với `curChoice` (vị trí con trỏ hiện tại) hay không.
        - Nếu trùng: Gọi `setHoveredState()` cho nút và nhãn tương ứng (làm nút đổi màu và phóng to).
        - Nếu không: Gọi `setNormalState()` (trạng thái bình thường).
    + **Vẽ nền:** Gọi `WINDOW.draw(panel)` để vẽ khung nền bảng cài đặt.

+ **Vẽ chi tiết:** Tiếp tục vòng lặp để vẽ lần lượt các `buttons[i]` và `labels[i]` đè lên trên nền.

2.1.2 void Menu::drawMenu()

- **Vị trí:** File Menu.cpp.
- **Chức năng:** Đây là hàm vẽ cơ bản cho hầu hết các menu. Nó hiển thị danh sách các lựa chọn dọc theo màn hình theo một khung có sẵn và xuyên suốt.
- **Cách hoạt động:**
  + **Xử lý Logic:** Hàm `if` kiểm tra (`isActive`) `EventController()`. Điều này có nghĩa là khi menu được vẽ, nó cũng đồng thời lắng nghe phím bấm để cập nhật con trỏ ngay lập tức.
  + **Hiệu ứng Hover:** Tương tự như setting, nó duyệt qua danh sách buttons và labels. Nếu `curChoice = i`, nó kích hoạt trạng thái `Hovered` (tô màu texture, phóng to), ngược lại là `Normal`.
  + **Render:** Thực hiện lệnh `WINDOW.draw` theo thứ tự: vẽ panel (nền) trước → vẽ buttons → vẽ labels.

2.1.3 void Menu::drawPopUpMenu()

- **Vị trí:** File Menu.cpp.
- **Chức năng:** Vẽ menu dạng cửa sổ nổi (pop-up), thường dùng cho menu tạm dừng hoặc kết thúc game. Có thể hiểu rằng menu được vẽ đè lên màn hình hiện tại.
- **Cách hoạt động:**
  + **Tạo lớp phủ:** Khởi tạo một hình chữ nhật `sf::RectangleShape overlay` có kích thước bằng toàn màn hình (`WINDOW.WIDTH, WINDOW.HEIGHT`).
  + **Làm tối nền:** Đặt màu cho overlay là `sf::Color(0, 0, 0, 150)`. Làm tối mờ bàn cờ phía sau đi, tạo hiệu ứng xóa phông.
  + **Vẽ lớp phủ:** Gọi `WINDOW.draw(overlay)` trước tiên.

+ **Vẽ Menu:** Gọi hàm `drawMenu()` để vẽ các nút bấm và khung menu đè lên trên lớp phủ đó.

2.1.4 `void Cell::draw()`

- **Vị trí:** File Board.cpp.
- **Chức năng:** Vẽ một ô cờ.
- **Cách hoạt động:**
  + **Kiểm tra dữ liệu:** Hàm kiểm tra biến `logicMark` của ô đó.
  + **Vẽ có điều kiện:**
    - Nếu `logicMark != '.'` (tức là ô đã có quân X hoặc O), nó mới thực hiện `WINDOW.draw(spriteMark).`
    - Nếu ô trống, hàm không làm gì cả.

2.1.5 `void Board::setHighlight()`

- **Vị trí:** File Board.cpp.
- **Chức năng:** Cập nhật vị trí và vẽ ô vuông màu để người chơi biết mình đang chọn ô nào trên bàn cờ.
- **Cách hoạt động:**
  + **Lấy vị trí:** Truy cập vào tọa độ pixel của ô hiện tại đang được chọn thông qua `cells[curX][curY].spriteMark.getPosition()`.
  + **Cập nhật:** Đặt vị trí của hình chữ nhật highlight trùng với vị trí vừa lấy được.
  + **Vẽ:** Gọi `WINDOW.draw(highlight)` để hiển thị khung viền lên màn hình.

2.1.6 `void Board::draw()`

- **Vị trí:** File Board.cpp.
- **Chức năng:** Hàm tổng hợp để vẽ toàn bộ bàn cờ hoàn chỉnh.
- **Cách hoạt động:**

+ **Vẽ nền:** Gọi `WINDOW.draw(board)` để vẽ hình ảnh bàn cờ.
+ **Vẽ các quân cờ:** Sử dụng hai vòng lặp lồng nhau để gọi `cells[i][j].draw()`.
+ **Vẽ con trỏ:** Cuối cùng, gọi `setHighlight()` để vẽ khung highlight của con trỏ đè lên trên cùng.

2.1.7 `void Game::drawResult()`

- **Vị trí:** File Game.cpp.
- **Chức năng:** Hiển thị thông báo kết quả trận đấu (thắng hoặc hòa) khi game kết thúc.
- **Cách hoạt động:**
  + **Xác định nội dung:** Dựa vào biến `gameResult` và `curPlayer`:
    - Nếu thắng (`gameResult = 1`): Chuỗi thông báo là "X WINS" hoặc "O WINS". `Texture` nền được set là `XWinsBackground` hoặc `OWinsBackground` tương ứng.
    - Nếu hòa: Chuỗi là "IT'S A DRAW", texture là `DrawBackground`.
  + **Căn chỉnh:** Dùng `setPosition` và `setOrigin` để thiết lập vị trí và tâm cho khung thông báo và chữ để chúng hiện giữa màn hình hoặc vị trí mình chỉ định.
  + **Vẽ:** Thực hiện `WINDOW.draw(box)` sau đó là `WINDOW.draw(message)`.

2.1.8 Hiệu ứng vẽ quân cờ

- **Struct:** `EffectX` và `EffectO` trong Board.h.
- **Cơ chế hoạt động:**
  + **Với quân X:** Không hiện ngay 2 nét chéo. Hệ thống vẽ lần lượt từng nét mô phỏng động tác viết tay. Biến `progress` tăng dần theo thời gian để kéo dài độ dài của thanh `Sprite`.
  + **Với quân O:** Sử dụng `sf::VertexArray` để vẽ một hình tròn khép kín dần

dần (từ 0 độ đến 360 độ). Biến `curAngle` được cập nhật mỗi khung hình để tạo hiệu ứng vẽ vòng tròn.

+ **Trạng thái:** Biến `isDrawingX` và `isDrawingO` trong `Board` kiểm soát việc khi nào bắt đầu và kết thúc hiệu ứng này.

2.1.9 Hiệu ứng chuyển cảnh xé giấy

- **Struct:** `EffectPaper` trong UI.h.
- **Cơ chế hoạt động:**
  + Được kích hoạt khi reset game hoặc chuyển màn chơi.
  + Sử dụng một texture hình vết xé di chuyển ngang màn hình, kết hợp với việc di chuyển hai nửa hình nền tách ra hai bên, tạo ảo giác tờ giấy ghi chép bị xé bỏ để bắt đầu trang mới.

2.1.10 Hiệu ứng chiến thắng và hòa

- **Hàm:** `startWaveEffect()` và `updateWinEffect()` trong Board.cpp.
- **Cơ chế hoạt động:**
  + Khi có người thắng, các quân cờ không liên quan sẽ mờ đi hoặc thu nhỏ lại.
  + Dãy 5 quân chiến thắng sẽ được làm nổi bật hoặc phóng to theo nhịp (wave) để người chơi dễ dàng nhận biết đường thắng.

C. NHÓM CÂU LỆNH LOGIC VÀ XỬ LÝ

1. NHÓM BIẾN, ĐỐI TƯỢNG VÀ THÔNG SỐ

Tên biến	**Vị trí**	**Chức năng**
Quản lý Game		

`Game GAME`	Global.h	Đối tượng quản lý toàn bộ vòng lặp trò chơi.
`MainScreen CARO`	Global.h	Đối tượng quản lý màn hình chính.
`Setting SETTING`	Global.h	Đối tượng quản lý màn hình cài đặt nổi (pop-up).
`SoundManager SOUND`	Global.h, Sound.h	Chứa tất cả biến âm thanh.
Dữ liệu Bàn cờ		
`vector<vector<Cell>> cells`	Board.h	Mảng 2 chiều lưu trữ dữ liệu toàn bộ các ô cờ.
`char logicMark`	Board.h	Ký tự lưu giá trị logic của ô '.', X, O. Dùng để tính thắng thua.
`int curX, curY`	Board.h	Tọa độ logic hiện tại của con trỏ trong bàn cờ trò chơi.
`int BOARD_SIZE`	Board.h	Kích thước bàn cờ.
`int numInRow`	Board.h	Số quân cờ cần để thắng.
Trạng thái Game		
`bool inGame`	Game.h	Biến đánh dấu, xác định có đang trong ván chơi hay không.
`bool inSession`	Game.h	Biến đánh dấu, xác định phiên chơi.

`bool gameOver`	Game.h	Biến đánh dấu, xác định game đã kết thúc chưa.
`bool pausing`	Game.h	Biến đánh dấu, xác định game có đang tạm dừng hay không.
`int curPlayer`	Game.h	Chỉ số xác định người chơi hiện tại. (0 là X, 1 là O)
`int gameMode`	Game.h	Chế độ chơi: 0 là PvP, 1 là PvE – đấu với máy.
`int gameResult`	Game.h	Kết quả trận đấu (1 là Thắng, 0 là Hòa).
`Player PLAYER[2]`	Game.h	Mảng lưu đối tượng của cấu trúc chứa thông tin 2 người chơi.
Dữ liệu Menu		
`int curChoice`	Menu.h, Setting.h	Chỉ số của nút đang được chọn trong menu.
`int numOptions`	Menu.h, Setting.h	Tổng số lượng lựa chọn trong một menu.
`bool isActive`	Menu.h	Biến đánh dấu menu có đang nhận tín hiệu điều khiển hay không.
`bool isOpen`	Menu.h	Biến đánh dấu menu có đang mở vòng lặp hiển thị hay không.
Dữ liệu Người chơi		

`char mark`	Player.h	Ký tự đại diện người chơi (‘X’ hoặc ‘O’).
`int score`	Player.h	Điểm số (được tính bằng số trận thắng).
`int winStreak`	Player.h	Chuỗi thắng liên tiếp.
`int moveCount`	Player.h	Số nước đã đi trong ván hiện tại.

2. NHÓM HÀM LOGIC VÀ XỬ LÝ

2.1 KHỞI TẠO VÀ CÀI ĐẶT

Tên hàm	**Vị trí**	**Chức năng**
`Setting::Setting()`	Setting.h	Khởi tạo danh sách nút và icon cho phần cài đặt.
`SoundManager::SoundManager()`	Sound.h	Tải tất cả file âm thanh (định dạng wav) vào bộ nhớ đệm và thiết lập volume.
`MainScreen::MainScreen()`	MainScreen.h	Thiết lập menu chính.
`Button::Button(int, int)`	UI.h	Khởi tạo kích thước cho một nút bấm.
`Label::Label(string)`	UI.h	Khởi tạo font chữ và nội dung cho nhãn văn bản.
`TextureManager::TextureManager()`	UI.h	Tải hình ảnh vào bộ nhớ.

`FontManager::FontManager()`	UI.h	Tải font chữ.
`Window::Window()`	UI.h	Tạo cửa sổ với kích thước 1920*1080.
`Menu::createBox(...)`	Menu.cpp	Thiết lập kích thước và vị trí khung menu.
`Menu::createMenu(...)`	Menu.cpp	Tính toán vị trí và tạo danh sách các nút/label trong menu.
`Board::Board()`	Board.h	Hàm tạo bàn cờ.
`Board::createBoard()`	Board.cpp	Thiết lập vị trí các ô cờ và lưới toạ độ.
`GameOption::GameOption()`	Game.h	Khởi tạo menu pause.
`GameOver::GameOver()`	Game.h	Khởi tạo menu kết thúc.
`GameMode::GameMode()`	Game.h	Khởi tạo menu chọn chế độ (PvP, PvE, Extreme).
`Game::Game()`	Game.h	Thiết lập quân cờ mặc định.

2.2 XỬ LÝ INPUT VÀ ĐIỀU KHIỂN

Tên hàm	**Vị trí**	**Chức năng**
`void`	Setting.cpp	Xử lý phím di chuyển và enter trong

`Setting::EventController()`		màn hình setting.
`void Menu::EventController()`	Menu.cpp	Vòng nhận đầu vào chung cho các menu.
`void Game::EventController()`	Game.cpp	Quan trọng: Xử lý phím chơi game. W/A/S/D di chuyển trên bàn cờ, Enter đánh dấu quân cờ, ESC để tạm dừng.
`void MainScreen::start()`	MainScreen.cpp	Vòng lặp chính của màn hình menu, vẽ và nhận đầu vào.
`int main()`	main.cpp	Hàm chính bắt đầu chương trình.

2.3 XỬ LÝ LOGIC VÀ TRẠNG THÁI

Tên hàm	**Vị trí**	**Chức năng**
`void Setting::moveCursor(int)`	Setting.cpp	Tính toán chỉ số nút đang chọn trong setting và phát âm thanh.
`void Menu::moveCursor(int)`	Menu.cpp	Tính toán chỉ số nút đang chọn trong menu và phát âm thanh.
`void MainScreen::selectOption()`	MainScreen.cpp	Logic khi chọn một mục ở menu chính.
`void GameOption::selectOption()`	Game.cpp	Logic xử lý nút trong menu pause.
`void`	Game.cpp	Logic xử lý nút trong menu Game

`GameOver::selectOption()`		Over.
`void GameMode::selectOption()`	Game.cpp	Lưu lựa chọn chế độ chơi vào biến `gameMode`.
`void GameMode::chooseGameMode()`	Game.cpp	Chạy vòng lặp để bắt buộc người dùng chọn chế độ chơi (PvP, PvE) trước khi vào game.
`void Board::moveCursor(...)`	Board.cpp	Cập nhật toạ độ `curX`, `curY` logic trên bàn cờ và kiểm tra biên có hợp lệ hay không.
`bool Board::isValidCell()`	Board.cpp	Kiểm tra ô hiện tại có trống hay không.
`bool Board::placeMark(...)`	Board.cpp	Gán giá trị ('X'/'O') và cập nhật texture cho ô cờ.
`bool Board::checkWin(...)`	Board.cpp	Thuật toán duyệt 4 hướng để kiểm tra 5 quân liên tiếp.
`bool Board::checkDraw()`	Board.cpp	Kiểm tra bàn cờ đã đầy chưa.
`void Board::resetData()`	Board.cpp	Xóa sạch bàn cờ về trạng thái ban đầu.
`void Game::start()`	Game.cpp	Bắt đầu quy trình chơi.
`void Game::pause()`	Game.cpp	Gán biến `pausing = true`.
`void Game::resume()`	Game.cpp	Gán biến `pausing = false`.
`void Game::quit()`	Game.cpp	Thoát vòng lặp game.

`void Game::stop()`	Game.cpp	Dừng game khi có kết quả.
`void Game::switchPlayer()`	Game.cpp	Đổi lượt người chơi.
`void Game::resetData()`	Game.cpp	Reset điểm số, lượt chơi.
`void Game::runGame()`	Game.cpp	Vòng lặp game chính: vẽ, nhận đầu vào và cập nhật trạng thái liên tục.

2.4 TRẠNG THÁI GIAO DIỆN

Tên Hàm	**Vị trí**	**Chức năng**
`void Button::setNormalState()`	UI.cpp	Đặt hình ảnh nút về mặc định và kích thước nguyên thủy.
`void Button::setHoveredState()`	UI.cpp	Đổi hình ảnh nút sang chọn và phóng to lên 10%.
`void Button::setPressedState()`	UI.cpp	Xử lý hiệu ứng nút khi chọn.
`void Label::setNormalState()`	UI.cpp	Đặt màu chữ về màu chủ đạo và kích thước nguyên thủy.
`void Label::setHoveredState()`	UI.cpp	Đổi màu chữ sang màu trắng và phóng to lên 10%.
`void Label::setPressedState()`	UI.cpp	Xử lý hiệu ứng chữ khi chọn.

`void Menu::setActive(bool state)`	Menu.cpp	Gán giá trị cho biến `isActive`. Biến này quyết định xem menu đó có nhận tín hiệu từ bàn phím hay không.
`void Menu::closeMenu()`	Menu.cpp	Tắt menu.
`void Menu::openMenu()`	Menu.cpp	Mở menu.

2.5 CHI TIẾT CÁCH THỨC HOẠT ĐỘNG NHỮNG HÀM PHỨC TẠP

2.5.1 Khởi tạo và cài đặt

Nhóm hàm này chịu trách nhiệm cấp phát bộ nhớ, tải tài nguyên và thiết lập giá trị ban đầu khi chương trình vừa chạy.

2.5.1.1 `SoundManager::SoundManager()`

- **Vị trí:** File Sound.h.
- **Chức năng:** Tải âm thanh và cài đặt âm lượng.
- **Cách hoạt động:**
  + Tải nhạc nền và cài đặt chế độ lặp để nhạc chạy mãi.
  + Tải các hiệu ứng âm thanh ngắn vào `sf::SoundBuffer`.
  + Gán `buffer` cho các đối tượng `sf::Sound` và thiết lập âm lượng mặc định là `50.0`.

2.5.1.2 `TextureManager::TextureManager()`

- **Vị trí:** File UI.h.
- **Chức năng:** Tải toàn bộ hình ảnh từ ổ cứng vào bộ nhớ RAM.
- **Cách hoạt động:** Gọi hàm `loadFromFile("đường_dẫn")` liên tiếp cho các biến `texture`.

2.5.1.3 `Menu::createBox(...)`

- **Vị trí:** File Menu.cpp.
- **Chức năng:** Tạo khung nền cho menu.
- **Cách hoạt động:** Thiết lập kích thước, đặt tâm về giữa, đặt vị trí và gán `texture` cho biến `panel`.

2.5.1.4 `Menu::createMenu(...)`

- **Vị trí:** File Menu.cpp.
- **Chức năng:** Tạo danh sách các nút bấm động dựa trên danh sách tên nút được truyền vào.
- **Cách hoạt động:**
  + Nhận tham số là vị trí bắt đầu (`startX, startY`), khoảng cách giữa các nút (`spacing`) và danh sách tên nút (`texts`).
  + Thay đổi kích thước vector `buttons` và `labels` cho khớp số lượng nút.
  + Dùng vòng lặp để gán vị trí (`setPosition`) cho từng nút theo công thức: `startY + i * spacing`.
  + Gán chuỗi ký tự (`setString`) cho các nhãn và căn giữa chúng vào nút bấm.

2.5.1.5 `Board::createBoard()`

- **Vị trí:** File Board.cpp.
- **Chức năng:** Tính toán vị trí hiển thị cho 225 ô cờ (15x15) trên màn hình.
- **Cách hoạt động:**
  + Cài đặt hình ảnh và vị trí cho bàn cờ lớn (`board`).
  + Dùng 2 vòng lặp lồng nhau (duyệt hàng `i` và cột `j` từ 0 đến `BOARD_SIZE`).
  + Tính toán tọa độ `x`, `y` cho từng ô dựa trên công thức: `START + CELL_SIZE/2 + chỉ_số * CELL_SIZE`.
  + Cập nhật vị trí này cho `spriteMark` của từng `Cell` để khi vẽ, chúng nằm đúng ô lưới.

2.5.2 Xử lý input và điều khiển

Nhóm hàm này là cầu nối giữa người dùng và game, nhận phím bấm và gọi các hàm logic tương ứng với từng thao tác phím bấm đã nhận vào.

2.5.2.1 `void Menu::EventController()`

- **Vị trí:** File Menu.cpp.
- **Chức năng:** Xử lý thao tác điều hướng trong các menu (Menu chính, Menu Pause).
- **Cách hoạt động:**
    + Nhận thao tác phím bấm trên bàn phím.
    + Nếu bấm **W** (lên): gọi `moveCursor(-1)`.
    + Nếu bấm **S** (xuống): gọi `moveCursor(1)`.
    + Nếu bấm **Enter**: gọi hàm ảo `selectOption()`.

2.5.2.2 `void Game::EventController()`

- **Vị trí:** File Game.cpp.
- **Chức năng:** Xử lý toàn bộ thao tác khi người chơi đang trong ván game.
- **Cách hoạt động:**
    + Sử dụng `WINDOW.pollEvent(event)` để lấy thao tác đầu vào từ hàng đợi.
    + Nếu bấm nút đóng cửa sổ (`sf::Event::Closed`): gọi `WINDOW.close()`.
    + Nếu bấm **ESC**: gọi `pause()` để dừng game và `GAME_OPTIONS.openMenu()` để mở menu cài đặt.
    + Nếu bấm **Enter**: gọi `BOARD.placeMark()`. Nếu đánh thành công, tiếp tục kiểm tra thắng (`checkWin`) hoặc hòa (`checkDraw`). Nếu có kết quả thì dừng game, ngược lại thì đổi lượt (`switchPlayer`).
    + Nếu bấm **W/A/S/D**: gọi `BOARD.moveCursor` với các tham số hướng di chuyển quân cờ tương ứng.

2.5.3 Xử lý logic và trạng thái

2.5.3.1 `void Board::moveCursor(const int& dx, const int& dy)`

- **Vị trí:** File Board.cpp.
- **Chức năng:** Di chuyển con trỏ chọn ô trên bàn cờ.
- **Cách hoạt động:**
  + Phát âm thanh di chuyển `SOUND.sfxBoardMove.play()`.
  + Kiểm tra điều kiện biên: Tính tọa độ mới `curX + dx và curY + dy`. Chỉ cập nhật nếu tọa độ mới nằm trong khoảng `[0, BOARD_SIZE)`.
  + Cập nhật biến `curX`, `curY` nếu hợp lệ.

2.5.3.2 `bool Board::placeMark(const char& mark)`

- **Vị trí:** File Board.cpp.
- **Chức năng:** Đánh một quân cờ vào ô hiện tại.
- **Cách hoạt động:**
  + Gọi `isValidCell()`: Kiểm tra ô `cells[curX][curY]` có đang chứa ký tự hay không.
  + Nếu hợp lệ:
    - Phát âm thanh `sfxPlaceMark`.
    - Gán biến logic: `logicMark = mark (‘X’ hoặc ‘O’)`.
    - Cập nhật hình ảnh: `spriteMark.setTexture` thành hình X hoặc O tương ứng.
    - Trả về `true`.
  + Nếu không hợp lệ, trả về `false`.

2.5.3.3 `bool Board::checkWin(const char& mark)`

- **Vị trí:** File Board.cpp.

- **Chức năng:** Thuật toán kiểm tra điều kiện.
- **Cách hoạt động:**
    + Duyệt qua toàn bộ bàn cờ bằng 2 vòng lặp.
    + Tại mỗi ô, thực hiện 4 vòng lặp con để kiểm tra 4 hướng:
        - **Ngang:** Kiểm tra các ô `[i][j+k]`.
        - **Dọc:** Kiểm tra các ô `[i+k][j]`.
        - **Chéo chính:** kiểm tra các ô `[i+k][j+k]`.
        - **Chéo phụ:** kiểm tra các ô `[i-k][j+k]`.
    + Nếu tìm thấy chuỗi 5 ô liên tiếp có `logicMark` trùng với quân cờ đang xét, trả về `true`.

2.5.3.4 `bool Board::checkDraw()`

- **Vị trí:** File Board.cpp.
- **Chức năng:** Kiểm tra hòa.
- **Cách hoạt động:** Duyệt toàn bộ bàn cờ, nếu không còn ô nào trống thì trả về `true`.

2.5.3.5 `void Board::resetData()`

- **Vị trí:** File Board.cpp.
- **Chức năng:** Xóa bàn cờ.
- **Cách hoạt động:** Đặt con trỏ về giữa bàn cờ. Đưa bàn cờ về trạng thái trống.

2.5.3.6 `void Game::switchPlayer()`

- **Vị trí:** File Game.cpp.
- **Chức năng:** Chuyển lượt chơi giữa 2 người.
- **Cách hoạt động:**
    + Sử dụng phép toán XOR: `curPlayer ^= 1`.
    + Nếu `curPlayer` đang là `0` (00), XOR `1` thành 1 (01).

+ Nếu `curPlayer` đang là `1` (01), XOR `1` thành 0 (00).

2.5.3.7 `void Game::resetData()`

- **Vị trí:** File Game.cpp.
- **Chức năng:** Reset dữ liệu game.
- **Cách hoạt động:** Đặt người chơi về `0` (`'X'`), reset số nước đi. Nếu là ván mới hoàn toàn `(!inSession)` thì reset điểm số.

2.5.3.8 `void Game::runGame()`

- **Vị trí:** File Game.cpp.
- **Chức năng:** Vòng lặp chính điều phối toàn bộ ván chơi.
- **Cách hoạt động:**
  + Gọi `resetData()` để xóa bàn cờ và điểm số.
  + Đánh dấu `inGame = true`.
  + Vào vòng:
    - Xóa màn hình `WINDOW.clear`.
    - Nếu chưa `gameOver` và chưa `pausing`, gọi `EventController` để nhận phím bấm.
    - Vẽ bàn cờ `BOARD.draw()`.
    - Nếu `pausing`: vẽ menu tùy chọn đè lên.
    - Nếu `gameOver`: vẽ kết quả và menu kết thúc đè lên.
    - Cuối cùng gọi `WINDOW.display()` để hiển thị khung hình mới.

2.5.3.9 `void Bot::makeMove() & int Bot::minimax(...)`

- **Vị trí:** `Bot.cpp`.
- **Chức năng:** Xử lý trí tuệ nhân tạo (AI).

- **Cách hoạt động:**
    - Sử dụng thuật toán Minimax với độ sâu giới hạn `maxDepth = 5` để dự đoán nước đi.
    - Hàm `evaluateBoard` tính điểm lợi thế dựa trên số quân liên tiếp và vùng trống.
    - Hàm `getCandidateMoves` lọc các nước đi tiềm năng (gần các quân đã đánh) để tối ưu hiệu suất.

2.5.3.10 void GameController::saveGame(bool) & loadGame(string)

- **Vị trí:** Game.cpp.
- **Chức năng:** Lưu và khôi phục trạng thái game.
- **Cách hoạt động:**
    - **Lưu:** Ghi dữ liệu vào file text trong thư mục saved_games/. Cấu trúc ghi bao gồm: Lượt chơi hiện tại (`curPlayer`) → Toàn bộ ma trận bàn cờ (`cells`).
    - **Tải:** Đọc từng ký tự từ file và map lại vào mảng `cells` của `Board`, đồng thời cập nhật lại hiển thị (`spriteMark`).

2.5.3.11 Logic Extreme Game (Chế độ mở rộng)

- **Vị trí:** `ExtremeGame struct` trong `Game.cpp`.
- **Chức năng:** Quản lý bàn cờ 3x3 (9 bàn cờ coni 3x3).
- **Cách hoạt động:**
    - Sử dụng mảng `Board BOARD[3][3].`
    - Khi người chơi đánh vào một ô ở bàn cờ con, nước đi tiếp theo sẽ bị buộc phải đánh vào bàn cờ con tương ứng với vị trí ô vừa đánh (Logic bàn cờ lớn).

2.5.4 Trạng thái giao diện

2.5.4.1 `void Button::setNormalState()`

- **Vị trí:** File UI.cpp.
- **Chức năng:** Đưa nút bấm về trạng thái mặc định.
- **Cách hoạt động:**
    + Gán hình ảnh (`Texture`) của nút về `TEXTURE.Button`.
    + Đặt tỷ lệ kích thước về (`1`, `1`) là kích thước nguyên bản của nút.

2.5.4.2 `void Button::setHoveredState()`

- **Vị trí:** File UI.cpp.
- **Chức năng:** Đưa nút bấm về trạng thái đang chọn.
- **Cách hoạt động:**
    + Đổi hình ảnh sang `TEXTURE.ButtonHovered`.
    + Phóng to nút lên một chút với tỷ lệ (`1.1, 1.1`), lớn hơn 10% để tạo điểm nhấn thị giác.

2.5.4.3 `void Label::setNormalState()`

- **Vị trí:** File UI.cpp.
- **Chức năng:** Đưa dòng chữ (`Text`) trên nút về trạng thái mặc định.
- **Cách hoạt động:**
    + Đặt màu chữ (`setFillColor`) thành màu chủ đạo `COLOR.PRIMARY`.
    + Đặt tỷ lệ kích thước về (`1`, `1`).

2.5.4.4 `void Label::setHoveredState()`

- **Vị trí:** File UI.cpp.
- **Chức năng:** Đưa dòng chữ về trạng thái đang chọn.
- **Cách hoạt động:**
    + Đổi màu chữ sang màu trắng.

+ Phóng to chữ lên (`1.1`, `1.1`) đồng bộ với nút bấm.

2.6 Thuật toán BOT đánh cờ Caro

Hệ thống Bot được xây dựng dựa trên thuật toán Minimax kết hợp Cắt tỉa Alpha-Beta (Alpha-Beta Pruning) để tối ưu hóa hiệu suất tính toán. Thay vì duyệt ngẫu nhiên, Bot có khả năng đoán trước nhiều nước đi để chọn phương án tối ưu nhất.

2.6.1 Cấu trúc dữ liệu

- **Struct Bot (Bot.h):** Chứa các tham số quan trọng như `maxDepth = 5` (độ sâu tìm kiếm tối đa) và `maxRange = 2` (phạm vi tìm kiếm cục bộ).
- **Struct Move:** Lưu tọa độ (`x, y`) và điểm số `score` của nước đi đó để so sánh.

2.6.2 Quy trình xử lý

2.6.2.1 Sàng lọc nước đi

- **Vấn đề:** Bàn cờ 15x15 có 225 ô, nếu duyệt hết sẽ gây tràn bộ nhớ và lag game.
- **Giải pháp:** Hàm này chỉ chọn ra các ô trống nằm trong phạm vi lân cận (bán kính 2 ô) so với các quân cờ đã đánh. Điều này giúp thu hẹp không gian tìm kiếm xuống đáng kể.

2.6.2.2 Hàm đánh giá cục bộ

- Bot quét bàn cờ để chấm điểm cho trạng thái hiện tại.
- Cơ chế tính điểm dựa trên độ nguy hiểm:
  + Chuỗi 4 quân: Điểm rất cao (ví dụ: `100,000`).
  + Chuỗi 3 quân thoáng 2 đầu: Điểm cao.
  + Bị chặn đầu: Điểm giảm.
- Điểm tổng = (Điểm tấn công của Bot) - (Điểm phòng thủ chặn Người chơi).

2.6.2.3 Thuật toán Minimax

- Sử dụng đệ quy để giả lập các nước đi tương lai.

- **Lượt Max (Bot):** Tìm nước đi có điểm cao nhất để thắng.
- **Lượt Min (Người chơi):** Giả định người chơi sẽ đánh nước đi tốt nhất để chặn Bot (tìm điểm thấp nhất cho Bot).
- **Cắt tỉa Alpha-Beta:** Ngắt ngay nhánh tìm kiếm nếu phát hiện một nước đi tồi tệ hơn nước đi đã tìm thấy trước đó, giúp giảm thời gian tính toán từ vài giây xuống mili-giây.

2.7 Chế độ Extreme

Chế độ Extreme là một biến thể nâng cao, yêu cầu cấu trúc dữ liệu phức tạp hơn so với chế độ Classic.

2.7.1 Kiến trúc bàn cờ

- Thay vì một mảng 2 chiều đơn giản, `struct ExtremeGame` quản lý một mảng 2 chiều các đối tượng Bàn cờ: `Board BOARD[3][3].`
- Tổng cộng có 9 bàn cờ con hoạt động độc lập nhưng được hiển thị cùng lúc trên màn hình.

2.7.2 Logic xử lý và hiển thị

- **Quản lý tọa độ:** Hệ thống phải xử lý hai lớp tọa độ: Tọa độ của bàn cờ con (0-2, 0-2) và tọa độ của ô cờ bên trong bàn cờ đó.
- **Vòng lặp game riêng biệt:** `ExtremeGame` kế thừa từ `GameController` nhưng ghi đè các hàm quan trọng như `runGame, drawBoards, EventController` để phù hợp với luật chơi 9 bàn cờ.

2.8 Hệ thống save/load

Chức năng này đảm bảo tính toàn vẹn dữ liệu cho trò chơi, cho phép người chơi gián đoạn và tiếp tục bất cứ lúc nào.

2.8.1 Kỹ thuật quản lý file

- Sử dụng thư viện `C++17 <filesystem>` để duyệt thư mục `saves/` và lấy danh sách

file tự động.

- Giao diện Load game hiển thị danh sách file dưới dạng các nút bấm, kèm theo thông tin chi tiết như: Tên file, Chế độ chơi (PvP/PvE/Extreme), và Thời gian chỉnh sửa cuối cùng.

2.8.2 Quy trình tuần tự hóa dữ liệu

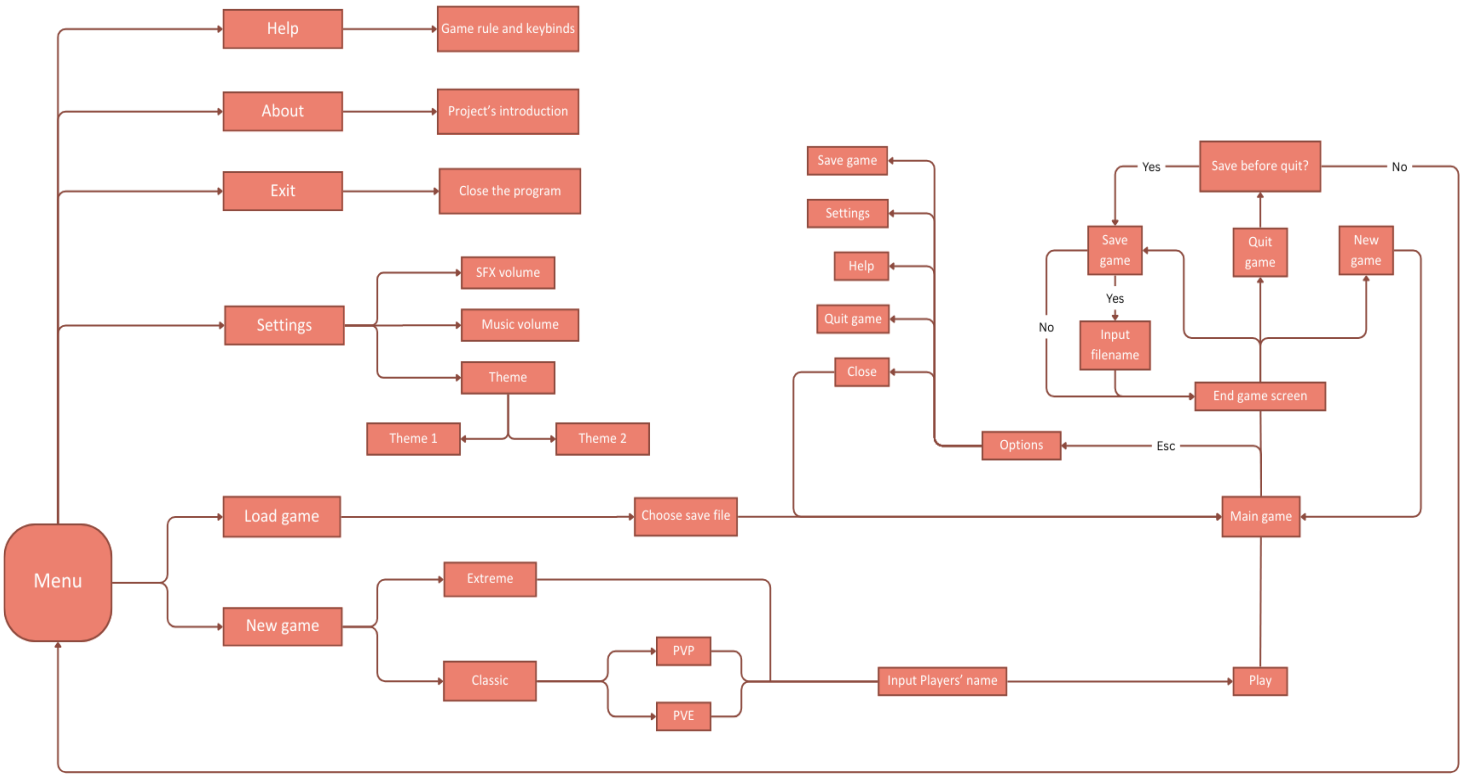
- Hàm `saveGame` thực hiện ghi trạng thái game ra file `.txt`.
- **Cấu trúc file lưu trữ:**
    + **Header:** Lưu thông tin stats (Chế độ chơi, lượt người chơi hiện tại `curPlayer`).
    + **Body:** Lưu ma trận bàn cờ. Sử dụng 2 vòng lặp lồng nhau để duyệt toàn bộ `cells[i][j]` và ghi ký tự logic (`X, O, .`) vào file.

2.8.3 Quy trình giải tuần tự hóa

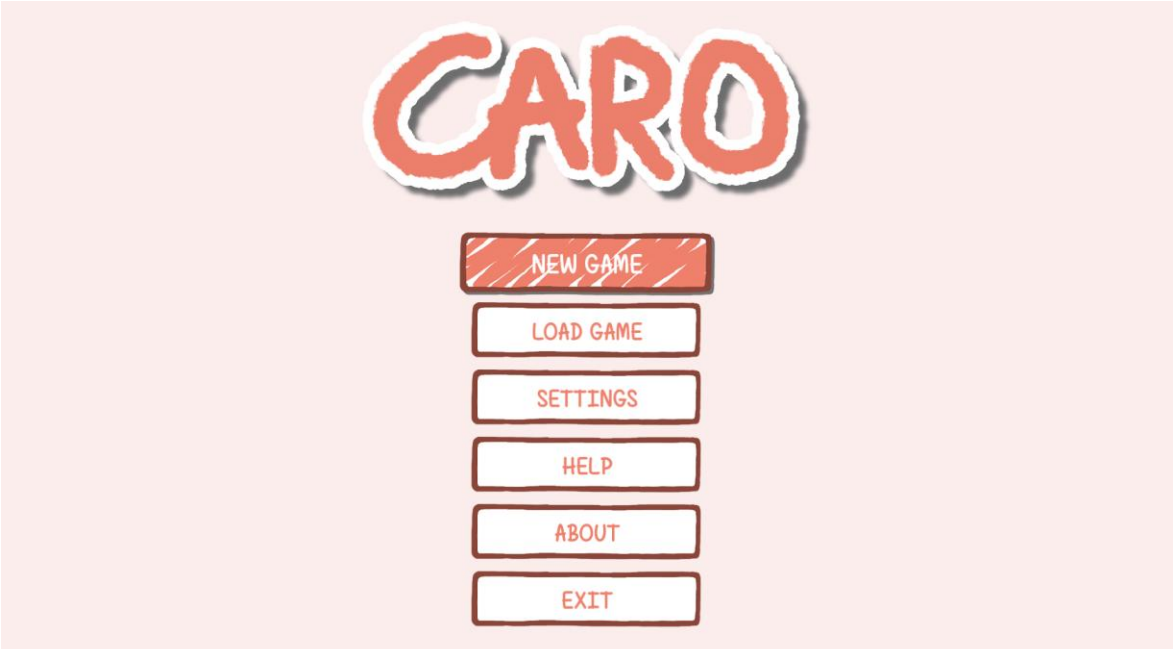
- Hàm `loadGame` đọc file theo quy trình ngược lại.
- **Quan trọng:** Sau khi đọc ký tự logic (`'X'/'O'`) từ file, hàm phải gọi lại logic đồ họa (`setTexture`) để cập nhật lại hình ảnh hiển thị trên bàn cờ, đảm bảo dữ liệu bộ nhớ và hình ảnh khớp nhau 100%.
- **Xử lý lỗi:** Kiểm tra file có tồn tại hay bị lỗi định dạng không trước khi load để tránh crash game.

D. MÔ PHỎNG CÁCH TRÒ CHƠI HOẠT ĐỘNG

1. LƯU ĐỒ TOÀN BỘ CHƯƠNG TRÌNH



2. MÀN HÌNH CHÍNH

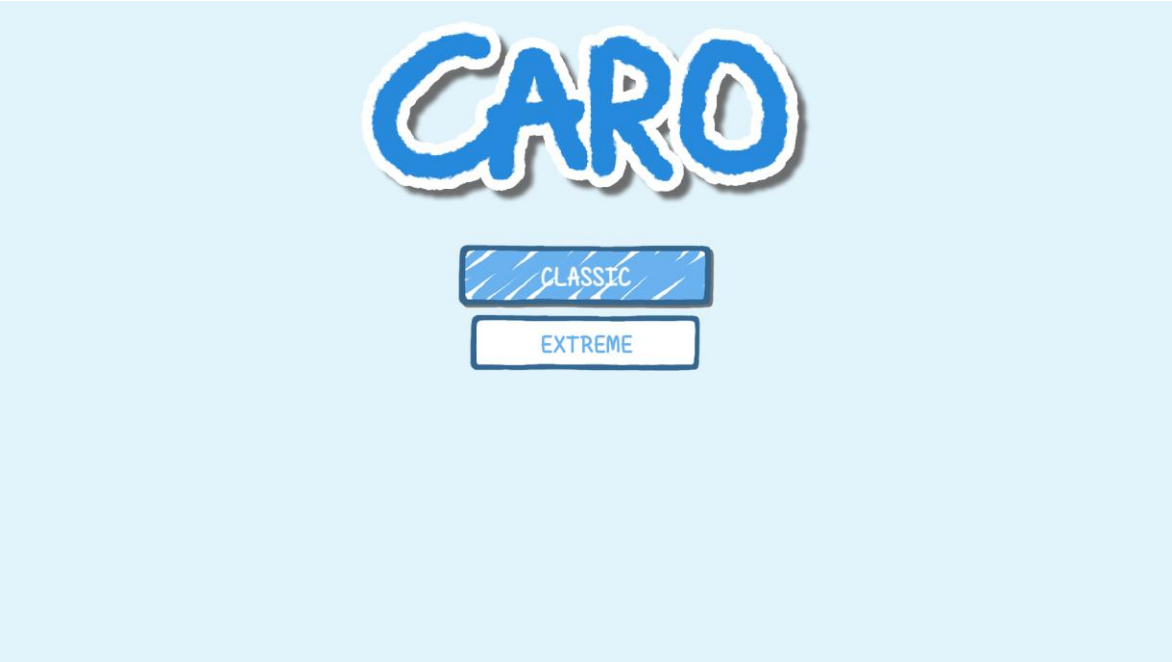


Hiện tại ở màn hình chính có 6 nút bấm NEW GAME, LOAD GAME, SETTINGS, HELP, ABOUT và EXIT . Để bắt đầu tạo mới một ván chơi, ta di chuyển con trỏ về nút NEW GAME và nhấn phím **Enter**. Để thoát trò chơi ta di chuyển con trỏ về nút EXIT và nhấn phím Enter. Nút LOAD GAME dùng để khởi tạo lại ván chơi được người chơi lưu lại từ trước. Nút ABOUT dùng để xem thông tin người hướng dẫn, thông tin các thành viên nhóm và công cụ sủ dụng.

Màn hình SETTINGS gồm các chức năng như tắt nhạc nền, âm thanh hiệu ứng và thay đổi bộ màu chủ đạo.



Sau khi nhấn NEW GAME thì chương trình chuyển sang màn hình chọn chế độ chơi CLASSIC và EXTREME. Chọn Classic thì tiếp tục có thêm màn hình chọn chế độ PvP và PvE.



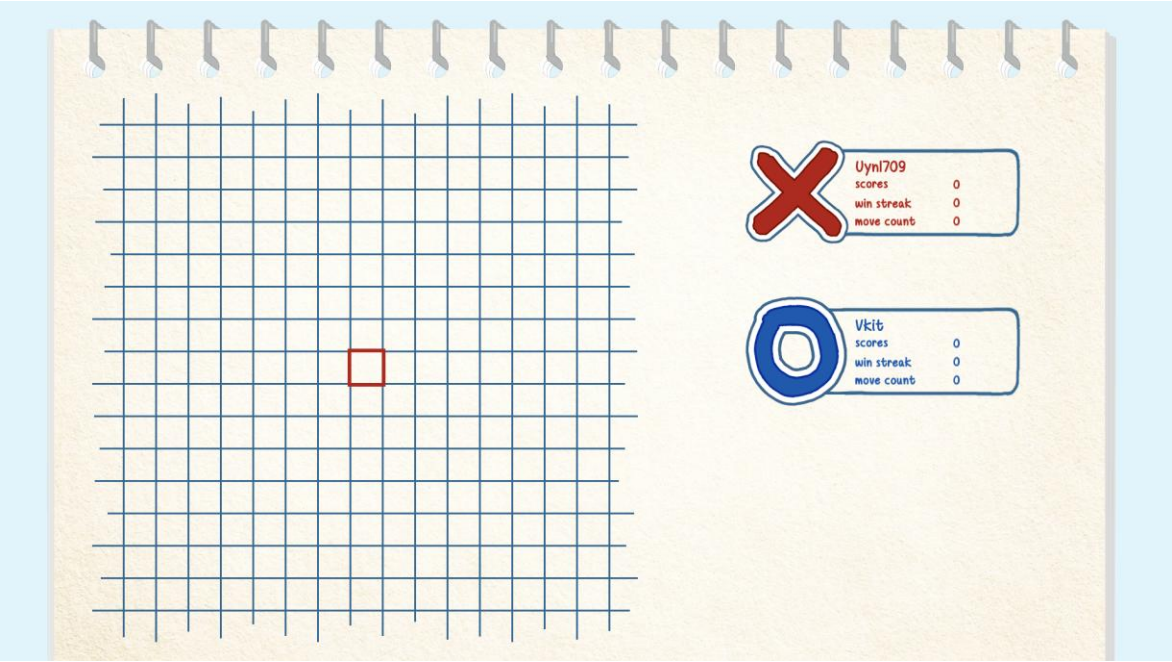
Trước khi bắt đầu một ván chơi mới ta đều được đặt tên cho 2 người chơi, hoặc đặt tên cho bot và người chơi.

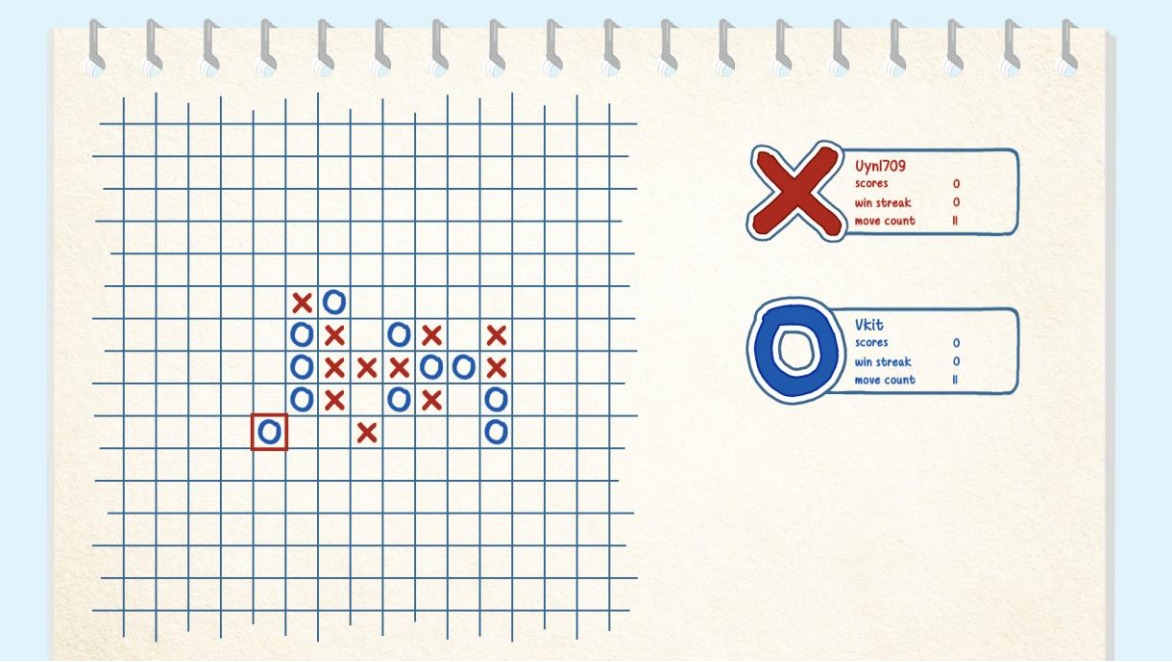


3. MÀN HÌNH TRÒ CHƠI

Sau khi đã chọn chế độ chơi thì chương trình sẽ chuyển sang màn hình bàn cờ như ảnh dưới đây. Người chơi tới lượt của mình sẽ sử dụng phím **W/S/A/D** để di chuyển theo các

hướng **lên/xuống/trái/phải** tương ứng. Con trỏ sẽ được tô màu tương phản với bàn cờ, và trùng màu với lượt đi của người chơi hiện tại, thuận tiện cho việc quan sát.

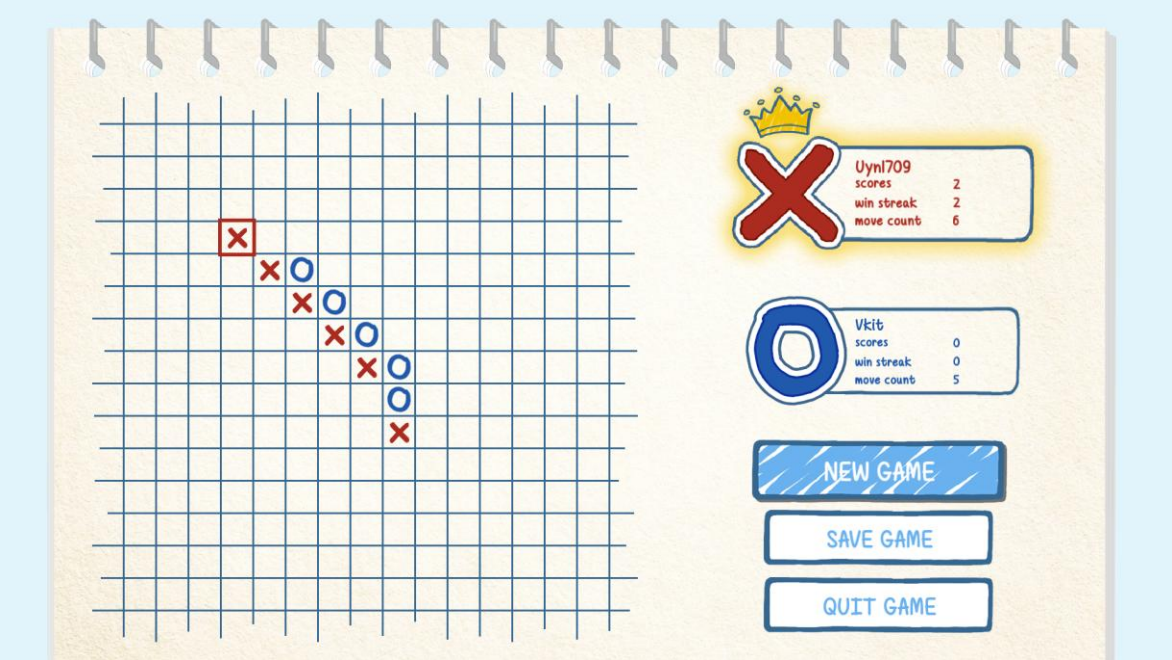




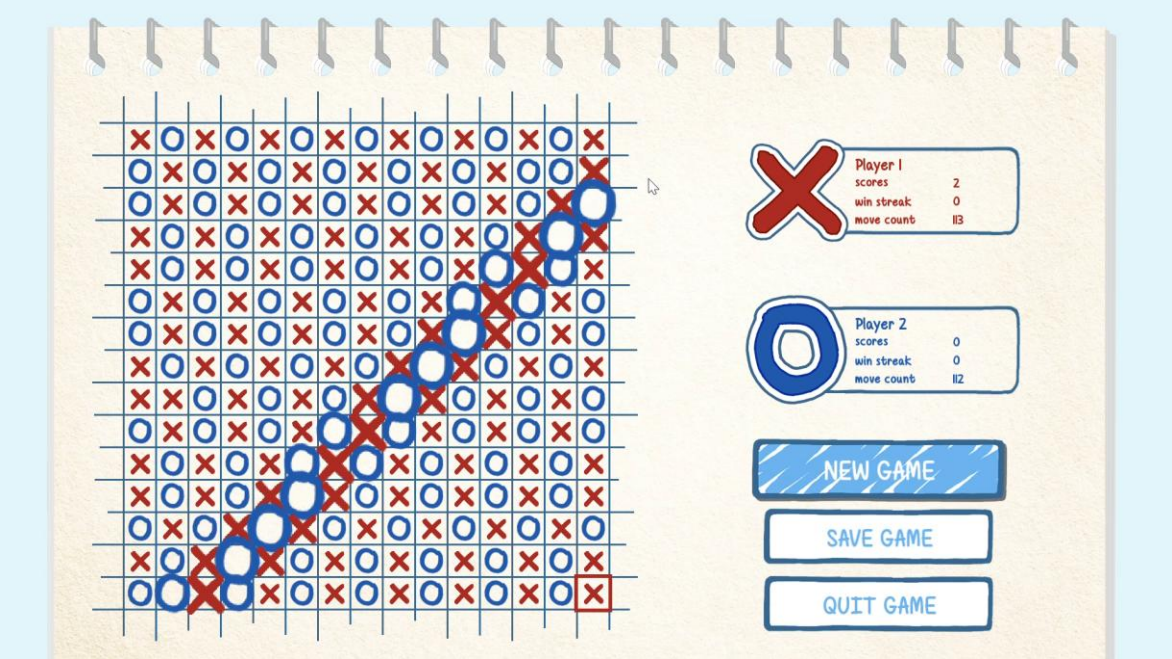
Phía bên phải là hai ô hiển thị thông số của từng người chơi bao gồm số trận thắng, số trận thắng liên tiếp và số lượt đi trong một ván đấu.

Đây là màn hình hiển thị thông báo thắng cuộc sau khi đã kiểm tra hợp lệ logic chiến

thắng.



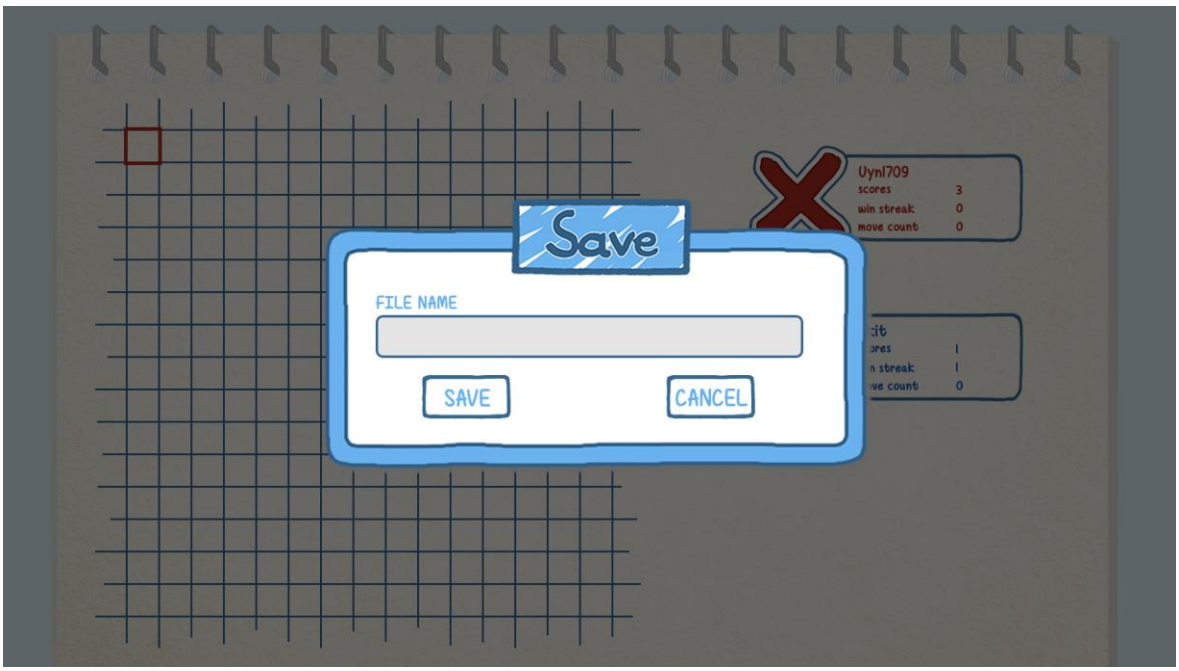
Đây là màn hình hiển thị thông báo thắng cuộc hòa sau khi bàn cờ đã được lấp đầy nhưng không có người chiến thắng.



Trạng thái sau khi có người chiến thắng hoặc hòa gọi là GameOver. Lúc này màn hình sẽ

hiện thông báo chiến thắng/hòa và menu lựa chọn gồm NEW GAME (bắt đầu ván chơi mới), SAVE GAME và QUIT GAME.

Khi chọn SAVE GAME, chương trình sẽ hiển thị khung cho người chơi đặt tên, thuận tiện cho việc phân biệt các ván chơi với nhau.

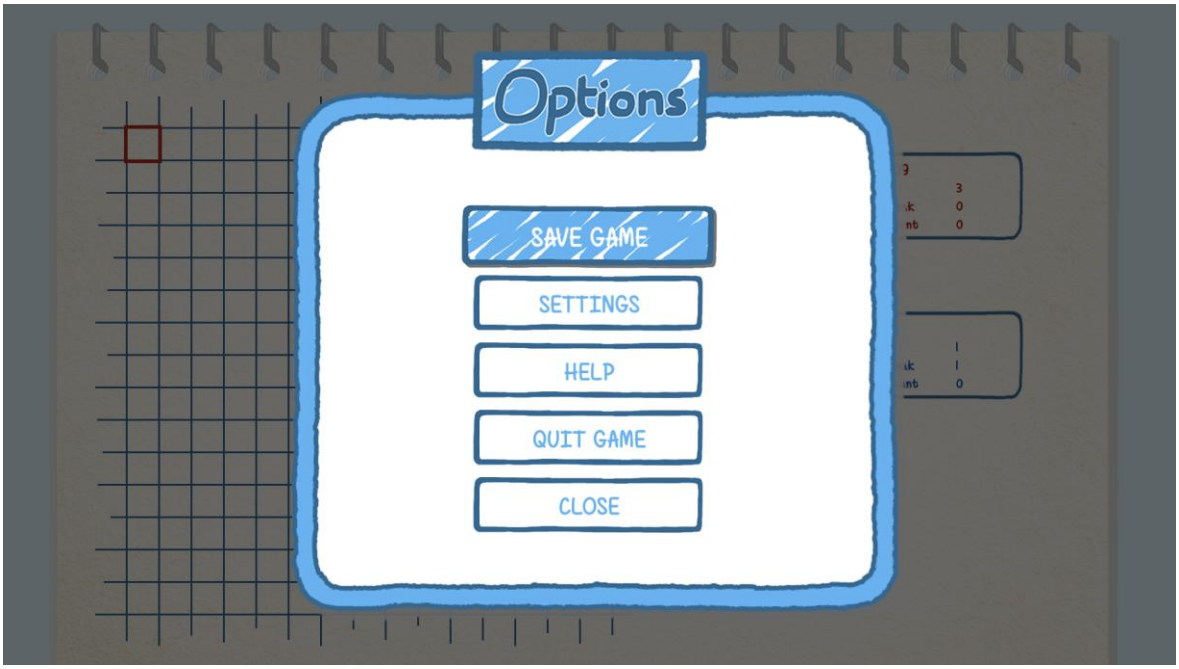




Màn hình LOAD GAME ngoài màn hình chính sẽ hiển thị cùng lúc 3 phiên bản ván đấu đã được người chơi lưu trước đó. Chọn nút LOAD để khởi tạo ván đấu.

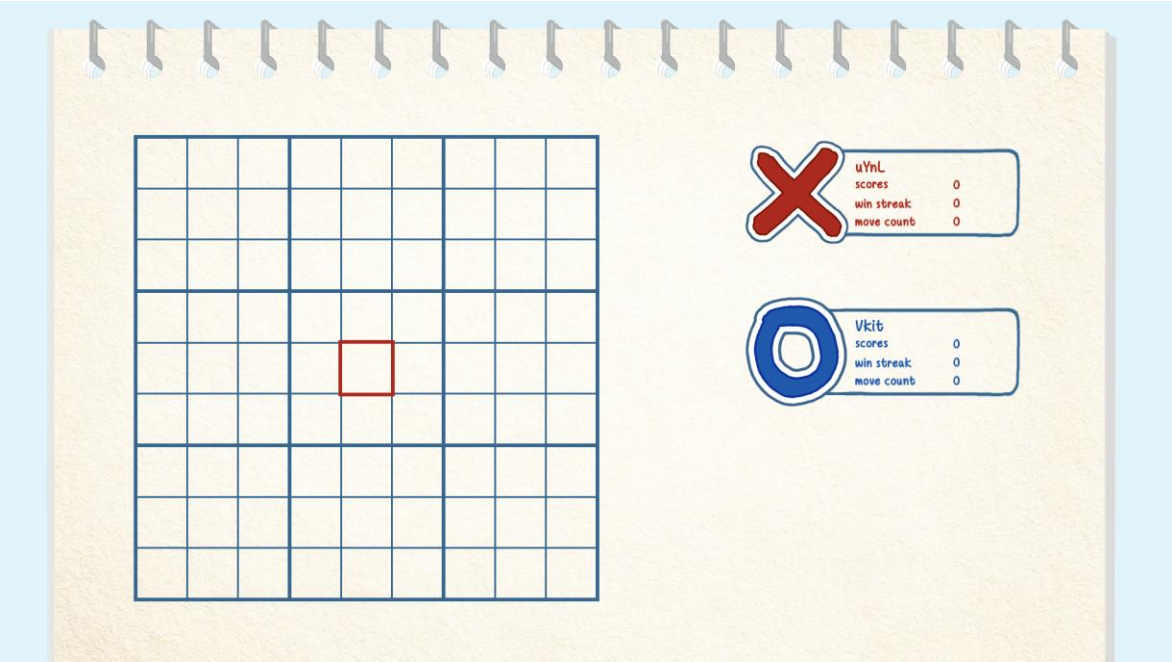
4. MÀN HÌNH OPTIONS (TRONG MÀN HÌNH TRÒ CHƠI)

Khi nhấn nút ESC trên bàn phím thì game sẽ mở lên bảng Opstion gồm các lựa chọn SAVE GAME và SETTINGS, HELP giống với ngoài màn hình chính, QUIT GAME quay về màn hình chính và CLOSE để tắt màn hình Options.

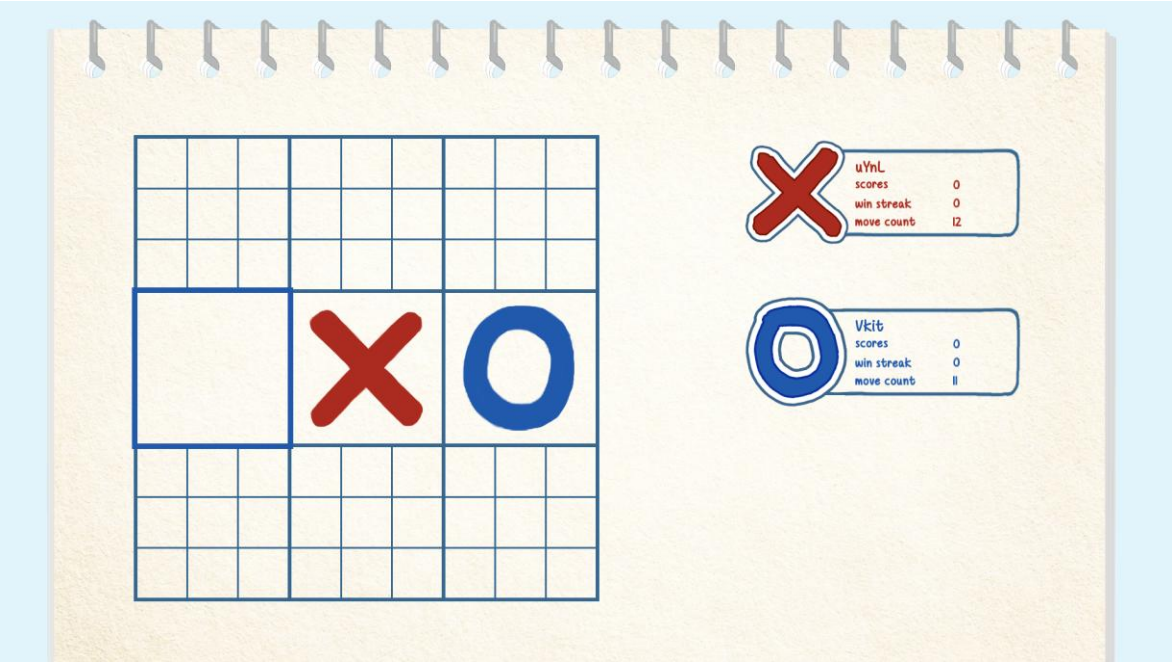


5. CHẾ ĐỘ EXTREME

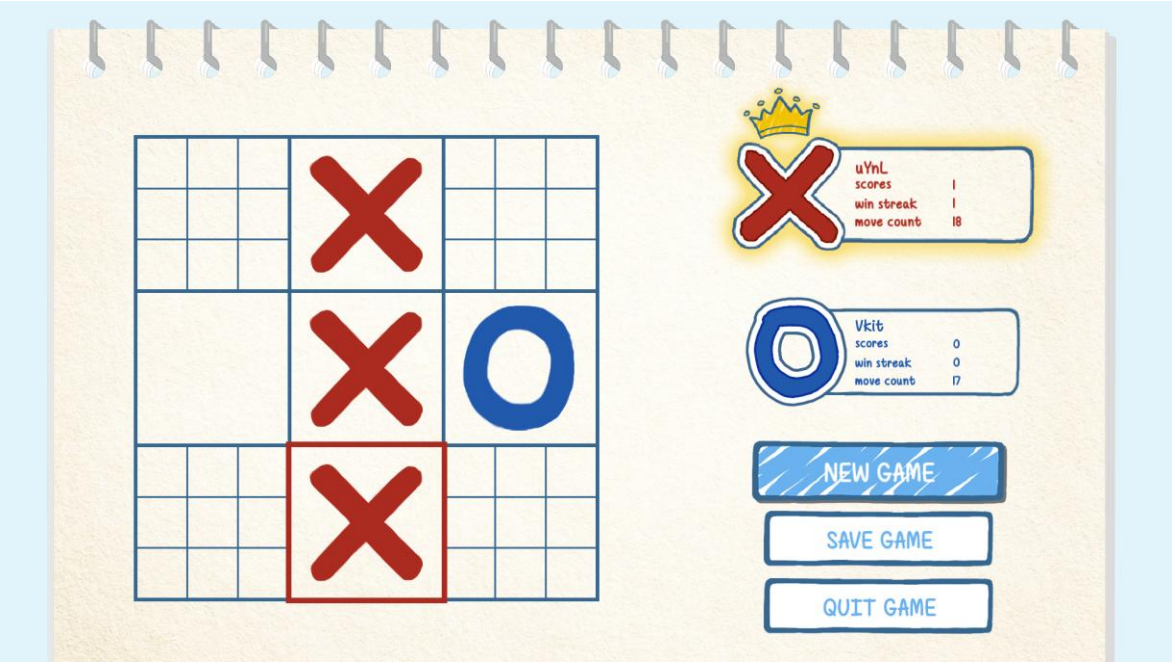
Bao gồm 9 ô cờ lớn, mỗi ô cờ lớn gồm 3 ô cờ nhỏ



Luật chơi tương tự game Caro 3x3. Sau khi chiến thắng sẽ được đánh một dấu lớn ở ô cờ lớn, hòa sẽ tô trắng.

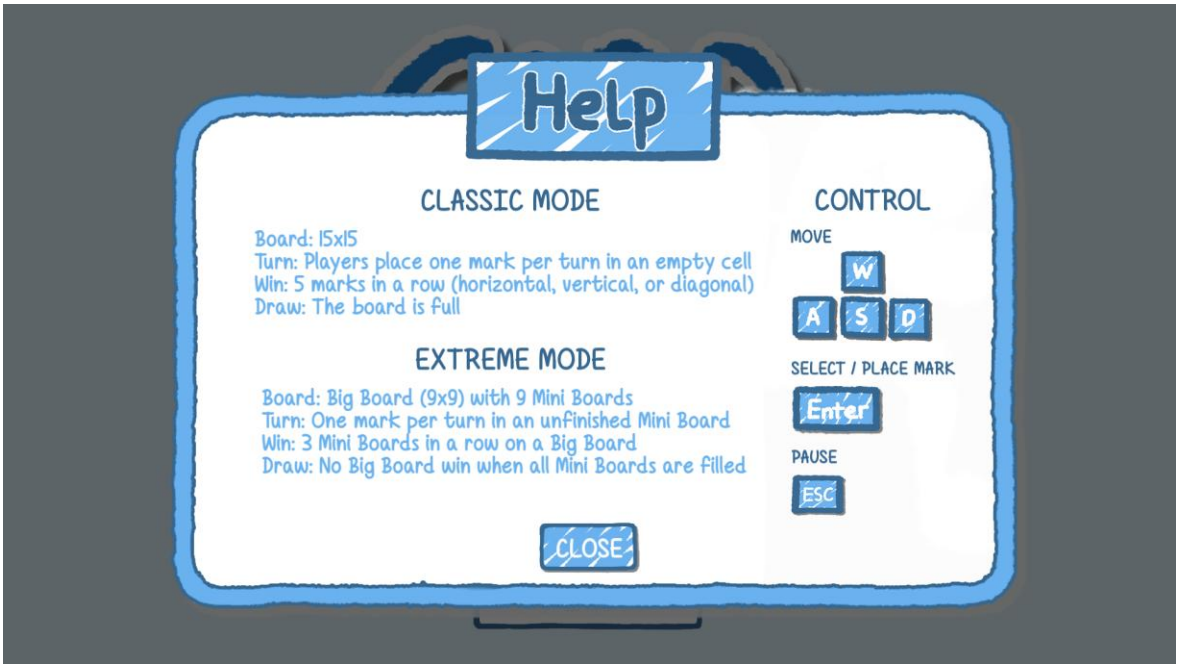


Tích lũy đủ các dấu ở các ô yêu cầu sẽ giành chiến thắng.



6. Màn hình HELP và ABOUT

Màn hình HELP chứa các chỉ dẫn thao tác điều khiển cần thiết và mô tả về các chế độ chơi có trong chương trình.



Màn hình ABOUT chứa thông tin về người hướng dẫn, thành viên nhóm – liên hệ và

nguồn tài liệu sử dụng.

